ASSIGNMENT 5 – CO5

PART A | TASK A3

DISCOVER THE DATA-ACCESS DESIGN PATTERN

Generic DAO Design for Student, Course and Marks Data

Course: Programming in Java
Topic: Data Persistence Engineering – Files and Databases

Design report covering the repeated database-access workflow, a generic Dao interface, a StudentDao design, object-to-table mapping, and comparison with SQL embedded in service or GUI classes.

1. Problem Understanding

The scenario contains three classes: StudentDB, CourseDB and MarksDB. Each class opens a database connection, builds an SQL string, executes it, converts ResultSet rows into objects, and closes the connection. Because these steps are repeated, the design contains duplicated data-access logic. The task is to identify the common workflow, separate the entity-specific parts, and define a reusable Data Access Object (DAO) abstraction.

2. A3.1 – Identify the Common Pattern

Repeated steps in all three classes

Repeated step	Purpose
Open / obtain connection	Establish access to the relational database.
Prepare SQL	Construct the SQL operation for the required entity/table.
Execute SQL	Run insert, select, update or delete using JDBC.
Process ResultSet	Read returned columns and create Java objects where rows are returned.
Close JDBC resources	Release ResultSet, Statement/PreparedStatement and Connection resources.

Parts that differ

The entity/table name differs (student, course or marks); the columns and their Java data types differ; primary-key types may differ; SQL parameter values differ; and the mapping from ResultSet columns to the corresponding Java class differs. Validation rules and entity-specific business rules may also differ. The connection, resource-management and general CRUD workflow should not be copied into every entity-specific class.

3. A3.2 – Design a Generic Data-Access Interface

A DAO (Data Access Object) isolates persistence logic from the rest of the application. A generic interface Dao uses T for the entity type and K for its primary-key type. The same contract can therefore be reused for Student, Course and Marks entities.

Generic interface design (Java-style interface declaration)

```
public interface Dao {
    void insert(T entity) throws SQLException;
    Optional findById(K id) throws SQLException;
    List findAll() throws SQLException;
    boolean update(T entity) throws SQLException;
    boolean delete(K id) throws SQLException;
}
```

The declaration above is an interface design example, not a complete runnable application. It assumes java.sql.SQLException, java.util.List and java.util.Optional are imported. A concrete DAO supplies the SQL and row-mapping logic for its entity.

Example: StudentDao implements Dao

```
public class StudentDao implements Dao {
    private final Connection connection;

    public StudentDao(Connection connection) {
        this.connection = connection;
    }

    @Override
    public void insert(Student student) throws SQLException {
        String sql = "INSERT INTO student (student_id, name, email) VALUES (?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, student.getStudentId());
```

```
            ps.setString(2, student.getName());
            ps.setString(3, student.getEmail());
            ps.executeUpdate();
        }
    }

    @Override
    public Optional findById(Integer id) throws SQLException {
        String sql = "SELECT student_id, name, email FROM student WHERE student_id = ?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    Student s = new Student(
                        rs.getInt("student_id"),
                        rs.getString("name"),
                        rs.getString("email")
                    );
                    return Optional.of(s);
                }
            }
        }
        return Optional.empty();
    }

    // findAll(), update() and delete() follow the same JDBC pattern:
    // entity-specific SQL + PreparedStatement + ResultSet/object mapping.
}
```

The snippet demonstrates insert and findById in detail. To satisfy the complete interface, StudentDao must also implement findAll(), update() and delete(). They use the same resource management pattern and bind values with PreparedStatement. The interface specifies the contract; each DAO provides its table-specific implementation.

4. A3.3 – Map Objects to Tables

For the Student example, assume the Student class contains studentId, name and email. These are mapped to the corresponding columns in the student table. The task does not prescribe specific Student fields or SQL column names; the mapping below is an illustrative, consistent design.

Java class field	Database column	Example type	Role
studentId	student_id	INT	Primary key
name	name	VARCHAR(100)	Student name
email	email	VARCHAR(150)	Student email

The student_id column is the primary key: it uniquely identifies each student row and is used by findById(id), update and delete. A corresponding SQL table could be defined as:

```
CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL
);
```

5. Alternative Concept – SQL Inside Service or GUI Classes

If SQL were written directly inside service or GUI classes, each class would contain its own connection handling, SQL statements, parameter binding, ResultSet conversion and resource closing. When a table column is renamed, every SQL statement and every row-mapping location that refers to that column may need to be found and updated. This creates scattered persistence logic and increases the risk of inconsistent changes.

With a DAO design, SQL and table-to-object mapping are concentrated in the relevant DAO implementation. A column rename normally requires changes in that DAO (and possibly the entity mapping/schema migration), rather than changes across multiple GUI and service classes. The service or GUI depends on the DAO interface and is less coupled to database details.

Aspect	Generic DAO design	SQL in service / GUI
Column rename	Usually update the affected DAO SQL/mapping and schema migration.	Update every scattered SQL and mapping reference.
Maintainability	Centralised persistence logic; less duplication.	Duplicated logic and stronger coupling.
Time per DB operation	Same underlying SQL/database work; abstraction does not change asymptotic cost.	Same underlying SQL/database work.
Space per operation	Same order of memory for connection, statement and result data.	Same order of memory for connection, statement and result data.

Complexity analysis

For either design, the database operation's time complexity is determined mainly by the SQL query, indexes, number of rows accessed and database execution plan—not by whether the SQL is placed in a DAO or a GUI/service class. For example, a primary-key lookup is typically O(log n) with a B-tree index (or expected O(1) with a suitable hash index), while a full-table retrieval is O(n). Both designs also have the same asymptotic space needs for the returned objects and JDBC resources. DAO improves separation of concerns and maintenance; it does not inherently make the SQL query faster.

6. Conclusion

StudentDB, CourseDB and MarksDB repeat the same JDBC workflow, while their entity-specific SQL and object mappings differ. A generic Dao interface captures common CRUD operations and allows StudentDao to implement

the persistence details for Student. Mapping Java fields to table columns makes the relationship explicit. Compared with SQL embedded in GUI or service classes, DAO centralises database changes and reduces coupling, while leaving the underlying time and space complexity of database operations essentially unchanged.

Submission note

Task A3 is in Part A – Persistence Design & Pseudocode. It asks for a design/interface example and object–table mapping; it does not request a complete runnable Java application. The Java-style snippets are illustrative design evidence, not a full project to execute.